

A COMPLETE PRACTITIONER'S GUIDE

Advanced Retrieval- Augmented Generation

From First Principles to Production-Grade
Agentic AI Systems — with LangChain, LangGraph & Gemini

MODULE TWO

Document Processing & Chunking

Beginner to Industry Level • Assumes only basic Python

MODULE TWO

Document Processing and Chunking

The retriever can only find what the indexing pipeline put into the knowledge base — and the indexing pipeline is only as good as how it loads and splits your documents. This module is a deep, practical treatment of the first half of every RAG system: turning messy real-world files into clean, well-sized, well-labelled chunks. You will master every LangChain document loader you need, every text-splitting strategy and exactly when to use it, the science of choosing chunk size and overlap, the discipline of metadata management, and how to assemble all of it into a production-grade ingestion pipeline.

WHERE WE ARE

In Module 1 you learned that a RAG system runs in two phases — an offline **indexing phase** (load → split → embed → store) and an online **query phase**. This module zooms all the way into the first two steps of indexing: **load** and **split**. Embedding and storage are Modules 3 and 4. Get this module right and everything downstream becomes easier; get it wrong and no clever retriever can save you.

2.1 Introduction

In Module 1 we glossed over loading and splitting in a single line of code each, because the goal there was to see the whole RAG loop at once. That was a deliberate simplification. In any real project, **document processing is where most of the engineering effort — and most of the bugs — actually live.**

The reason is simple: the world's knowledge does not arrive as clean text. It arrives as PDFs with two-column layouts and scanned pages, as HTML pages buried in navigation menus and advertisements, as CSV exports, JSON API responses, Word documents, Markdown wikis, and source code. Before any of that can be embedded and retrieved, it must be (1) **loaded** — extracted into plain text with useful metadata attached — and (2) **split** — broken into chunks of the right size and shape.

These two operations sound mundane. They are not. The single most common reason a RAG system gives bad answers is not a weak language model and not a weak retriever —

it is **bad chunking upstream**. If a chunk is too large, it dilutes meaning and wastes the context window. If it is too small, it loses the surrounding context needed to make sense of a fact. If a chunk is split in the wrong place, a single idea is severed across two chunks and neither one is retrievable on its own. The retriever can only ever return the chunks you created; if those chunks are badly formed, the retriever is doomed before it starts.

This module gives document processing the serious treatment it deserves. We will cover every loader and every splitter you will realistically need, explain the internal mechanics so you can reason about them, derive practical rules for chunk size and overlap, and end by building a reusable, production-grade ingestion pipeline.

2.2 Why This Topic Matters

WHY THIS MATTERS

There is a phrase that has followed data systems for decades: *garbage in, garbage out*. In RAG it is doubly true. The indexing pipeline is the "in." Every embedding, every similarity search, every generated answer is built on top of the chunks this module teaches you to create. An hour spent improving your chunking strategy will almost always improve answer quality more than an hour spent on any other single component. Senior RAG engineers know this; it is why "how do you decide your chunk size?" is a standard interview question.

Four concrete reasons this module repays close study:

1. **Chunking quality sets a ceiling on retrieval quality.** A retriever cannot return a well-formed answer if no well-formed chunk exists. You are literally building the search index's atoms.
2. **Loaders determine what knowledge even enters the system.** A loader that silently drops tables, skips scanned pages, or pulls in webpage navigation junk corrupts the knowledge base at its source — and these failures are invisible until a user asks the wrong question.
3. **Metadata is the foundation of trust and control.** Source citations, filtering by document type or date, and access control all depend on metadata that must be attached *during* ingestion. You cannot add it later.
4. **It is where most RAG bugs originate.** When debugging a production RAG system, experienced engineers inspect the chunks first. Knowing how chunks were made is how you fix the system.

2.3 A Beginner-Friendly Explanation

Let us build intuition before any code.

Think of the indexing phase as preparing a library so it can be searched. You begin with a pile of raw materials — books, magazines, printed emails, spreadsheets. Two jobs must happen before anyone can search them.

Job one: get the words out of the containers. A PDF is a *container*; the knowledge is the *text inside it*. A web page is a container wrapped in menus and ads; the knowledge is the article in the middle. **Loading** is the job of opening each container and extracting the actual text — and, importantly, noting where it came from (which book, which page). In LangChain, the tools that do this are called **document loaders**.

Job two: cut the text into searchable pieces. Imagine trying to find one fact by being handed an entire 400-page book. Useless. You want to be handed the single relevant paragraph. **Splitting** (also called **chunking**) is the job of cutting each long document into small, self-contained passages — *chunks* — each focused enough that, when retrieved on its own, it still makes sense and still answers a question. In LangChain, the tools that do this are called **text splitters**.

So the mental model for this module is just two verbs: **load** (open the containers, get the text + where-it-came-from) and **split** (cut the text into bite-sized, meaningful chunks). Everything else is detail about doing those two verbs well for different kinds of input.

DEFINITION

A **document loader** is a component that reads a data source (a PDF, a web page, a CSV file, and so on) and converts it into one or more standardised **Document** objects, each containing extracted text plus metadata about its origin.

A **text splitter** (or **chunker**) is a component that takes long Document objects and divides them into smaller chunks suitable for embedding and retrieval.

A **chunk** is one such small passage — the atomic unit that gets embedded, stored, retrieved, and fed to the LLM.

2.4 Real-World Analogy: Preparing Index Cards for a Library

REAL-WORLD ANALOGY

Picture an old-fashioned library researcher building a card catalogue. They take each book and write its key ideas onto small index cards — one idea per card. They cannot write a whole chapter on one card (too much to absorb at a glance), and they will not write half a sentence on a card (too little to be meaningful on its own). They aim for the *Goldilocks size*: one self-contained idea per card.

They also do something clever: when an idea spills across the bottom of one card onto the next, they **repeat the last sentence** at the top of the new card, so neither card loses the thread. That repetition is **chunk overlap**.

Finally, on the corner of every card they write the book title and page number. That corner note is **metadata** — it is how, weeks later, anyone can trace a fact back to its source.

This analogy maps cleanly onto the module:

In the analogy	In a RAG ingestion pipeline
Opening the book to read its text	The document loader extracting text
Writing ideas onto index cards	The text splitter creating chunks
One self-contained idea per card	Choosing the right chunk size
Repeating a sentence across cards	Chunk overlap
Title and page number in the corner	Metadata attached to each chunk
The finished box of index cards	The set of chunks ready for embedding

Hold this picture; we will return to it when deriving chunk-size rules.

2.5 Core Concepts: The Ingestion Pipeline

Document processing is the front of the **ingestion pipeline** — the offline sequence that turns raw files into a searchable knowledge base. Here is the full pipeline with this module's scope highlighted:

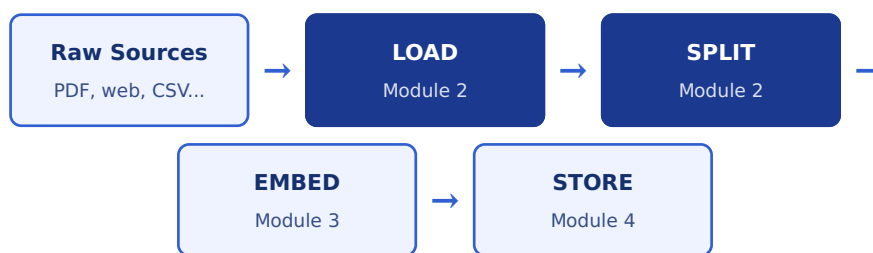


Figure 2.1 — The ingestion pipeline. This module owns the two shaded stages: loading and splitting.

Two ideas underpin everything that follows.

The `Document` object is LangChain's universal currency. No matter what the original source was — a PDF, a website, a database row — a loader converts it into the same standardised structure: a `Document`. Every `Document` has exactly two important attributes:

- `page_content` — a string holding the extracted text.
- `metadata` — a Python dictionary holding information *about* that text (its source, page number, creation date, and anything else you choose to attach).

Because every loader outputs `Document` objects and every splitter consumes and produces `Document` objects, the entire pipeline speaks one language. You can swap a PDF loader for a web loader without changing anything downstream. This uniformity is the quiet design decision that makes LangChain pipelines composable.

Loading and splitting are separate concerns. Loading answers "*what text exists and where is it from?*" Splitting answers "*how do I cut that text into retrievable pieces?*" They are different problems with different tools, and you should think about them separately even though they run back to back.

2.6 Technical Deep Dive: Document Loaders

A **document loader** connects a data source to your pipeline. LangChain ships well over a hundred loaders; you only need a handful. Every loader exposes the same method, `.load()`, which returns a `List[Document]`. (Many also offer `.lazy_load()`, which yields documents one at a time — important for very large sources, discussed under best practices.)

2.6.1 Loading PDFs

PDF is the most common enterprise document format and, unfortunately, one of the messiest. A PDF describes where glyphs are *painted* on a page, not the logical reading order — which is why PDF text extraction is genuinely hard, especially for multi-column layouts, tables, and scanned (image-only) pages.

LangChain offers several PDF loaders with different trade-offs:

Loader	Backend	Strengths	Use when
<code>PyPDFLoader</code>	<code>pypdf</code>	Simple, fast, one Document per page	Standard, text-based PDFs (the default choice)
<code>PyMuPDFLoader</code>	<code>PyMuPDF</code>	Fastest, good layout handling, rich metadata	Speed matters, or layout is complex
<code>PDFPlumberLoader</code>	<code>pdfplumber</code>	Strong table extraction	The PDF is table-heavy
<code>UnstructuredPDFLoader</code>	<code>unstructured</code>	Detects titles, lists, tables as elements	You need structure-aware parsing
<code>AmazonTextractPDFLoader</code> etc.	OCR services	Reads scanned/image PDFs	The PDF is scanned (no embedded text)

For most of this book, `PyPDFLoader` is our default. It is reliable, has the fewest dependencies, and conveniently returns **one Document per page** with the page number already in metadata.

```
# pip install langchain-community pypdf
from langchain_community.document_loaders import PyPDFLoader

loader = PyPDFLoader("data/handbook.pdf")
docs = loader.load()  # list[Document], one per page

print(f"Pages loaded: {len(docs)}")
print("Page 1 metadata:", docs[0].metadata)
print("Page 1 text (first 200 chars):", docs[0].page_content[:200])
```

The `metadata` of each page will look something like `{'source': 'data/handbook.pdf', 'page': 0}`. That `source` and `page` are gold — they are exactly what you will later use to show citations.

COMMON MISTAKE

Assuming every PDF loader will extract text correctly. If you load a **scanned PDF** (a document that is really a photograph of pages) with `PyPDFLoader`, `.load()` succeeds without error but every `page_content` comes back **empty or near-empty** — there is no embedded text to extract. The pipeline then indexes nothing, and the bug only surfaces when users get "I don't know" for everything. **Always print the length of the loaded text right after loading.** If a scanned PDF is involved, you need an OCR-based loader instead.

2.6.2 Loading Web Pages

To build a chatbot over a website or online documentation, you load web pages. The workhorse is `WebBaseLoader`.

```
# pip install beautifulsoup4
from langchain_community.document_loaders import WebBaseLoader

loader = WebBaseLoader([
    "https://example.com/docs/getting-started",
    "https://example.com/docs/faq",
])
docs = loader.load()
print(f"Loaded {len(docs)} web pages.")
```

`WebBaseLoader` fetches each URL and uses BeautifulSoup to strip HTML tags down to text. But raw web text is noisy — it includes navigation bars, footers, cookie banners, and sidebars, none of which is real content. Indexing that noise pollutes your knowledge base.

The fix is to tell the loader which part of the page is the real content, using a BeautifulSoup filter:

```
import bs4
from langchain_community.document_loaders import WebBaseLoader

# Keep only the elements that hold real article content
loader = WebBaseLoader(
    web_paths=["https://example.com/docs/getting-started"],
    bs_kwargs={
        "parse_only": bs4.SoupStrainer(
            class_=("article-body", "main-content")
        )
    },
)
docs = loader.load()
```

Here `SoupStrainer` instructs BeautifulSoup to parse *only* HTML elements whose CSS class is `article-body` or `main-content`, discarding everything else. You discover the correct class names by opening the page in a browser and inspecting its HTML.

BEST PRACTICE

Never index a raw web page without filtering to its main content region. Navigation menus, footers, and ads create chunks that are semantically meaningless yet still get retrieved, crowding out real answers. A few minutes inspecting the page's HTML to find the right content selector pays for itself many times over. For large or JavaScript-rendered sites, consider sitemap-based loaders or a rendering loader such as one backed by Playwright.

2.6.3 Loading Structured Data (CSV, JSON)

Structured data — spreadsheets, database exports, API responses — needs loaders that respect its structure.

CSV files. `CSVLoader` reads a CSV and, by default, creates **one Document per row**, formatting that row's columns as `key: value` lines in `page_content`.


```
from langchain_community.document_loaders import CSVLoader

loader = CSVLoader(
    file_path="data/products.csv",
    metadata_columns=["product_id", "category"],  # promote these to metadata
)
docs = loader.load()
# Each Document is one product row; product_id & category are in metadata.
```

The `metadata_columns` argument is powerful: it lifts chosen columns out of the text and into structured metadata, so you can later *filter* by them (e.g. "search only within the `category = electronics` rows"). Metadata filtering is covered in Section 2.10.

JSON files. Use `JSONLoader`, which extracts content using a **jq-style query** that points at the fields you want.

```
# pip install jq
from langchain_community.document_loaders import JSONLoader

loader = JSONLoader(
    file_path="data/tickets.json",
    jq_schema=".[].body",  # extract the "body" field of each record
    text_content=True,
)
docs = loader.load()
```

The `jq_schema` `.[].body` means: for every element of the top-level array, take its `body` field. JSON shapes vary enormously, so the schema string is something you tailor to each file.

2.6.4 Other Loaders Worth Knowing

- `TextLoader` — plain `.txt` files.
- `Docx2txtLoader` / `UnstructuredWordDocumentLoader` — Microsoft Word documents.
- `DirectoryLoader` — loads every matching file in a folder, delegating to a per-file loader. Indispensable for the multi-document projects (multi-PDF RAG, enterprise knowledge assistant).
- `UnstructuredHTMLLoader` — local HTML files.
- `YoutubeLoader` — pulls a YouTube video's transcript (the foundation of the *YouTube Q&A Assistant* project).

```
# Load every PDF in a folder at once
from langchain_community.document_loaders import DirectoryLoader, PyPDFLoader

loader = DirectoryLoader(
    "data/",
    glob="**/*.pdf",  # all PDFs, including sub-folders
    loader_cls=PyPDFLoader,  # use PyPDFLoader for each file
    show_progress=True,
)
docs = loader.load()
```

INTERVIEW INSIGHT

A frequent practical question: "How would you build a knowledge base from a folder of mixed file types?" The strong answer mentions `DirectoryLoader` to walk the folder, choosing the right per-file loader by extension, and stresses two things: (1) **verify extraction succeeded** for each file — especially scanned PDFs — and (2) **attach consistent metadata** so every chunk's origin is traceable.

2.7 Technical Deep Dive: Text Splitting Strategies

Once documents are loaded, they must be split into chunks. First, *why* — then *how*.

2.7.1 Why Splitting Is Mandatory

You cannot simply embed a whole document. There are four hard reasons:

1. **The context window is finite.** The LLM can only read so much text at once. You retrieve several chunks per query; if each chunk were an entire document, they would not fit in the prompt.
2. **Retrieval precision collapses on large chunks.** An embedding is a *single* vector summarising an *entire* chunk. A chunk covering ten topics produces a blurry, averaged vector that matches no specific query well. A chunk covering one topic produces a sharp, specific vector. Small, focused chunks are precisely retrievable; large, mixed chunks are not.
3. **Embedding models have input limits and a quality sweet spot.** Most embedding models truncate input beyond a maximum length, and their representations are most faithful for short-to-medium passages.
4. **Cost and noise.** Stuffing huge chunks into the prompt costs more tokens and surrounds the real answer with irrelevant text that can distract the model.

WHY THIS MATTERS

The deepest reason for chunking is the *one chunk* → *one vector* rule. Retrieval finds chunks by comparing vectors. A vector is a fixed-size summary of meaning. If a chunk mixes many ideas, its vector is an average — a vague centroid that is "sort of near" many queries but a strong match for none. Splitting along natural idea boundaries produces vectors that each represent *one* clear meaning, and those are the vectors retrieval can find accurately. **Good chunking is really the craft of producing clean, single-meaning vectors.**

2.7.2 CharacterTextSplitter — The Simplest Splitter

`CharacterTextSplitter` splits text on a single separator (a blank line by default) and then merges the resulting pieces up to a target size.

```

from langchain_text_splitters import CharacterTextSplitter

splitter = CharacterTextSplitter(
    separator="\n\n",      # split on blank lines (paragraph breaks)
    chunk_size=1000,      # target chunk length in characters
    chunk_overlap=150,    # characters shared between adjacent chunks
)
chunks = splitter.split_documents(docs)

```

Its weakness: it knows only **one** separator. If a single "paragraph" is longer than `chunk_size`, it cannot break it gracefully — it will exceed the size or cut crudely. This is rarely the best choice. It exists mainly to understand the concept; in practice you reach for the recursive splitter.

2.7.3 RecursiveCharacterTextSplitter — The Default Choice

This is the splitter you will use most often, and the one to understand most deeply. It is *recursive* because it tries an **ordered list of separators**, falling back to the next one whenever a piece is still too large.

The default separator list is, in order: `["\n\n", "\n", " ", ""]` — paragraph break, then line break, then space, then individual character. The algorithm is:

1. Try to split the text on the **first** separator (paragraph breaks).
2. For any resulting piece still larger than `chunk_size`, split *that piece* on the **next** separator (line breaks).
3. Repeat down the list — words, then characters — until every piece fits.
4. Finally, **merge** adjacent small pieces back up toward `chunk_size`, applying the requested overlap.

The effect is that the splitter **respects natural text structure as much as possible**. It keeps whole paragraphs together when it can; only when a paragraph is too big does it descend to sentences, then words. It almost never cuts mid-word.

```

from langchain_text_splitters import RecursiveCharacterTextSplitter

splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000,
    chunk_overlap=150,
    separators=["\n\n", "\n", ". ", " ", ""], # customised order
)
chunks = splitter.split_documents(docs)
print(f"Created {len(chunks)} chunks.")

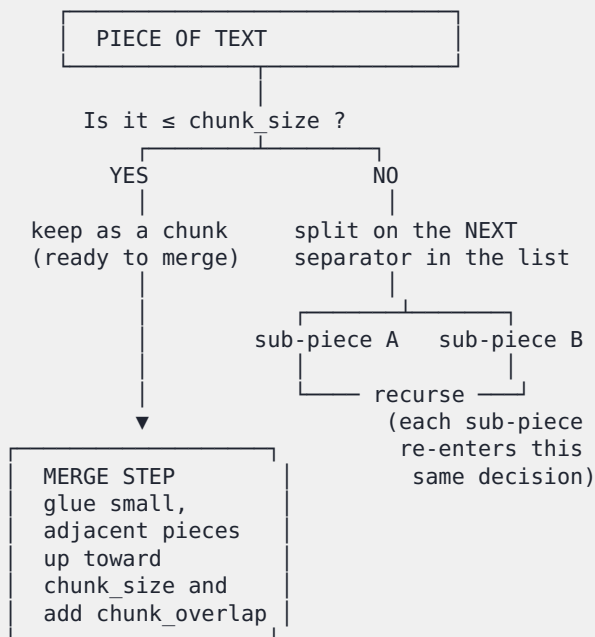
```

BEST PRACTICE

For ordinary prose — articles, manuals, reports, books — make `RecursiveCharacterTextSplitter` your default. It gives you the structure-awareness of paragraph splitting with a safe fallback for oversized passages, and it almost never produces broken-mid-word chunks. Start here; only switch to a specialised splitter (Markdown, code, semantic) when the content genuinely calls for it.

2.7.4 Internal Architecture: How the Recursive Splitter Decides

It helps to visualise the recursion as a decision the splitter makes for every piece of text:



The two-stage nature is the key insight: a **splitting** stage that recursively breaks text down at the best available boundary, followed by a **merging** stage that packs the resulting fragments back up to a useful size and stitches in the overlap. Understanding this is what lets you predict and debug the splitter's behaviour.

2.7.5 Markdown-Aware Splitting

Markdown documents (wikis, READMEs, technical docs) carry explicit structure in their headers (`#`, `##`, `###`). Splitting blindly on characters would throw that structure away. `MarkdownHeaderTextSplitter` instead splits **on headers** and records the header hierarchy of each chunk *into its metadata*.

```

from langchain_text_splitters import MarkdownHeaderTextSplitter

headers_to_split_on = [
    ("#", "Header 1"),
    ("##", "Header 2"),
    ("###", "Header 3"),
]
md_splitter = MarkdownHeaderTextSplitter(headers_to_split_on=headers_to_split_on)
chunks = md_splitter.split_text(markdown_string)

# Each chunk's metadata now contains the section it belongs to, e.g.:
# {'Header 1': 'Installation', 'Header 2': 'Linux'}

```

This is powerful for two reasons. First, chunks align with logical sections rather than arbitrary character counts. Second, every chunk *knows where it sits in the document* — that section context lands in metadata and can even be prepended to the chunk text so the embedding captures it. In practice you often run `MarkdownHeaderTextSplitter` first to split by section, then pass any over-long sections through `RecursiveCharacterTextSplitter` to enforce a size cap.

2.7.6 Code-Aware Splitting

Source code has its own structure — functions, classes, blocks. Splitting it like prose would slice a function in half. `RecursiveCharacterTextSplitter` has a `from_language` constructor that loads a separator list tuned to a specific programming language, so it prefers to break at function and class boundaries.

```

from langchain_text_splitters import RecursiveCharacterTextSplitter, Language

py_splitter = RecursiveCharacterTextSplitter.from_language(
    language=Language.PYTHON,
    chunk_size=800,
    chunk_overlap=100,
)
code_chunks = py_splitter.split_text(python_source_code)

```

`Language` supports Python, JavaScript, Java, C++, Go, Rust, Markdown, and many more. Code-aware splitting matters for documentation assistants and any "chat with this repository" tool — keeping each function intact means a retrieved chunk is a complete, runnable unit of logic rather than a fragment.

2.7.7 Semantic Splitting

Every splitter so far decides *where* to cut using **structure** (paragraphs, headers, syntax) or **size**. A **semantic splitter** decides where to cut using **meaning**. It walks through the document sentence by sentence, embeds each sentence, and places a chunk boundary wherever the meaning *shifts sharply* between consecutive sentences.

```
# pip install langchain-experimental
from langchain_experimental.text_splitter import SemanticChunker
from langchain_google_genai import GoogleGenerativeAIEmbeddings

embeddings = GoogleGenerativeAIEmbeddings(model="models/text-embedding-004")

semantic_splitter = SemanticChunker(
    embeddings,
    breakpoint_threshold_type="percentile",  # cut at the biggest meaning shifts
)
chunks = semantic_splitter.create_documents([long_text])
```

The appeal is that boundaries land exactly where topics genuinely change, producing maximally coherent, single-topic chunks. The cost is that splitting now requires embedding every sentence, which is slower and consumes API calls. Reserve semantic chunking for high-value corpora where retrieval quality justifies the extra ingestion cost; for most projects a well-tuned recursive splitter is the pragmatic choice.

2.7.8 Token-Based Splitting

`chunk_size` measured in *characters* is an approximation; LLMs and embedding models actually count **tokens** (sub-word units). When you must respect an exact token budget, split by tokens:

```
from langchain_text_splitters import RecursiveCharacterTextSplitter

splitter = RecursiveCharacterTextSplitter.from_tiktoken_encoder(
    chunk_size=256,  # 256 TOKENS, not characters
    chunk_overlap=32,
)
chunks = splitter.split_documents(docs)
```

A rough rule for English: **one token $\approx$ four characters $\approx$ three-quarters of a word**. A 1000-character chunk is therefore roughly 250 tokens. Token-based splitting matters most when you are tightly managing context-window limits or embedding-model input caps.

2.7.9 Choosing a Splitter — Summary Table

Content type	Recommended splitter	Why
General prose (articles, manuals, books)	<code>RecursiveCharacterTextSplitter</code>	Structure-aware with safe fallback
Markdown / wikis / technical docs	<code>MarkdownHeaderTextSplitter</code> (+ recursive)	Splits by section, captures hierarchy
Source code	<code>RecursiveCharacterTextSplitter.from_language</code>	Keeps functions and classes intact
High-value corpus, quality critical	<code>SemanticChunker</code>	Cuts exactly at topic boundaries
Strict token-budget control	Token-based recursive splitter	Counts the same units the model does
Quick prototype, uniform paragraphs	<code>CharacterTextSplitter</code>	Simplest; rarely the production choice

2.8 Chunking Best Practices: Size and Overlap

Two numbers dominate your splitter configuration: `chunk_size` and `chunk_overlap`. Choosing them well is part science, part measurement.

2.8.1 Choosing the Optimal Chunk Size

`chunk_size` is a genuine trade-off, and the index-card analogy captures it exactly:

- **Chunks too large** (e.g. 3000+ characters): each chunk mixes several ideas, so its embedding is a blurry average that matches no query precisely. Retrieval becomes imprecise and the prompt fills with off-topic text.
- **Chunks too small** (e.g. under 200 characters): each chunk loses the surrounding context that gives a fact meaning. A chunk reading *"It must be filed within 14 days."* is unhelpful if the chunk does not say *what* must be filed.
- **The sweet spot** holds **one self-contained idea with enough surrounding context to stand alone**.

There is no universal magic number, but these are sound, field-tested starting ranges:

Content / use case	Suggested chunk size (characters)	Notes
Short factual Q&A, FAQs	300 – 600	Small, sharp chunks for precise lookups
General documents, manuals, articles	800 – 1200	The reliable default range
Dense, narrative, or contextual text	1200 – 2000	Larger chunks preserve flowing argument
Source code	600 – 1000	Sized to keep functions whole

BEST PRACTICE

Do not agonise over the "perfect" chunk size in the abstract — **measure it**. Pick a sensible default (around 1000 characters), build the pipeline, and then run a set of representative test questions. If answers are vague and unfocused, your chunks are probably too large; if answers miss context and feel fragmented, they are probably too small. In Module 9 you will learn to make this rigorous with RAGAS metrics, turning chunk-size selection from guesswork into a measured experiment.

2.8.2 Chunk Overlap Strategies

Chunk overlap is the amount of text that consecutive chunks share. With `chunk_overlap=150`, the last 150 characters of chunk N are repeated as the first 150 characters of chunk $N+1$.

Overlap exists to solve the **boundary problem**. Without it, a sentence — or worse, a single idea spanning two sentences — can be sliced exactly at a chunk boundary, landing half in one chunk and half in the next, so that *neither* chunk fully contains the idea and *neither* is retrievable for a query about it. Overlap guarantees that text near a boundary appears, in full, in at least one chunk.

Practical guidance:

- A standard overlap is **10-20% of `chunk_size`** (e.g. 100-200 characters for a 1000-character chunk).
- **More overlap** improves boundary safety and continuity but **inflates the number of chunks**, increasing storage, embedding cost, and some retrieval redundancy.
- **Less overlap** is leaner but risks severing ideas at boundaries.
- 10-20% is the field-tested compromise; deviate only with a measured reason.

COMMON MISTAKE

Two opposite errors. The first is **zero overlap** — fast and lean, but it routinely guillotines ideas at chunk boundaries and quietly destroys recall. The second is **excessive overlap** (50%+) in the belief that "more context is always better." Heavy overlap balloons your chunk count, multiplies embedding cost, fills the retriever's results with near-duplicate chunks, and wastes context-window space on repetition. Stay in the 10-20% band unless measurement tells you otherwise.

2.9 Metadata Management and Filtering

Metadata is the most underrated topic in this module. Beginners treat it as an afterthought; senior engineers treat it as core architecture.

2.9.1 What Metadata Is and Why It Matters

Every `Document`'s `metadata` dictionary travels with its text through loading, splitting, embedding, and storage. Good metadata enables four capabilities that are otherwise impossible:

1. **Source citations.** `source` and `page` let the system answer "*according to handbook.pdf, page 14.*" Without them there is no traceability — and, as Module 1 stressed, no traceability means no trust.
2. **Metadata filtering.** You can restrict a search to chunks matching a condition — only this department, only documents newer than a date, only this file type. This is a *pre-filter* on the search space.
3. **Access control.** A `clearance` or `team` field lets the retriever return only chunks a given user is permitted to see — essential for any multi-tenant or enterprise system.
4. **Debugging and lifecycle management.** Knowing each chunk's origin lets you find, update, or delete everything from a specific document when it changes — without rebuilding the whole index.

2.9.2 Attaching and Enriching Metadata

Loaders attach basic metadata automatically (`source`, `page`). You should *enrich* it during ingestion with anything useful for filtering or citation later:

```
from datetime import date

for chunk in chunks:
    chunk.metadata["department"] = "human-resources"
    chunk.metadata["doc_type"] = "policy"
    chunk.metadata["ingested_on"] = str(date.today())
    chunk.metadata["language"] = "en"
```

BEST PRACTICE

Decide your metadata schema *before* you ingest, and apply it consistently to every chunk. Retrofitting metadata means re-ingesting the entire corpus. Keep values simple and filterable — strings, numbers, dates, booleans — because vector stores filter best on flat scalar fields. A small, deliberate, consistent metadata schema is one of the clearest signs of a professionally built RAG system.

2.9.3 Metadata Filtering at Query Time

Once chunks carry metadata, the retriever can filter on it. The filter is applied so that similarity search runs **only over the matching subset** of chunks:

```
# Search only within HR policy documents
retriever = vector_store.as_retriever(
    search_kwargs={
        "k": 4,
        "filter": {"department": "human-resources", "doc_type": "policy"},
    }
)
results = retriever.invoke("How many paid leave days do I get?")
```

This delivers two benefits at once: it improves **precision** (a query about HR cannot accidentally retrieve a finance chunk) and it improves **speed** (the search space is smaller). Filtering is the bridge between unstructured semantic search and structured database querying — and it is the foundation of the *self-query retriever* you will meet in Module 6, where the LLM generates these filters automatically from a natural-language question.

2.10 Workflow: A Production Ingestion Pipeline

Tying the module together, here is the shape of a robust, production-grade ingestion pipeline. It is the indexing phase done properly:

- | | |
|----------------|---|
| 1. DISCOVER | Walk source folders / URLs; list every document to ingest. |
| 2. LOAD | Pick the right loader per file type; extract text + metadata.
↳ VALIDATE: assert non-empty text; flag scanned/empty files. |
| 3. CLEAN | Strip boilerplate, fix encoding, normalise whitespace. |
| 4. SPLIT | Apply the appropriate splitter; enforce chunk_size & overlap. |
| 5. ENRICH | Attach the full metadata schema to every chunk. |
| 6. DEDUPLICATE | Drop exact-duplicate chunks (common across versioned docs). |
| 7. EMBED | Convert chunks to vectors — Module 3 |
| 8. STORE | Write vectors + text + metadata to the vector store — Module 4 |
| 9. PERSIST | Save the index; log counts, sources, timestamps for audit. |

The stages this module owns are 1-6. The principles that make this pipeline *production-grade* rather than a toy script are: **validate after loading** (catch empty extractions early), **process incrementally** (re-ingest only changed documents, not the whole corpus), **use `lazy_load()` for large sources** so you never hold everything in memory at once, and **log everything** (how many documents, how many chunks, from where, when) so the ingestion run is auditable.

WHY THIS MATTERS

A prototype ingestion script and a production ingestion pipeline differ not in the loaders and splitters they use — those are identical — but in **validation, incrementality, and observability**. The prototype assumes every file loads cleanly and re-processes everything every run. The production pipeline assumes files will be malformed, processes only what changed, and records exactly what it did. Interviewers probe this gap with questions like "how would you handle a corpus that updates daily?"

2.11 Step-by-Step Implementation and Code

We now build a complete, reusable ingestion module that loads a folder of PDFs, splits them, enriches metadata, validates the result, and reports statistics. This is `ingest.py` — a file you will genuinely reuse in later projects.

```

"""
ingest.py
A reusable, production-minded ingestion pipeline for RAG.
Stage covered: DISCOVER -> LOAD -> VALIDATE -> SPLIT -> ENRICH -> REPORT.
Embedding and storage (Modules 3-4) plug in at the end.
"""

from datetime import date
from pathlib import Path

from langchain_community.document_loaders import DirectoryLoader, PyPDFLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_core.documents import Document

def load_documents(source_dir: str) -> list[Document]:
    """Stage 1-2: discover and load every PDF in a folder."""
    loader = DirectoryLoader(
        source_dir,
        glob="**/*.pdf",          # every PDF, sub-folders included
        loader_cls=PyPDFLoader,   # one Document per page
        show_progress=True,
    )
    documents = loader.load()
    print(f"[LOAD] Loaded {len(documents)} pages from '{source_dir}'.")
    return documents

def validate_documents(documents: list[Document]) -> list[Document]:
    """Stage 2 (validate): drop pages whose text extraction failed."""
    valid, empty = [], 0
    for doc in documents:
        if doc.page_content and doc.page_content.strip():
            valid.append(doc)
        else:
            empty += 1
    if empty:
        print(f"[VALIDATE] WARNING: {empty} empty pages dropped "
              f"(possible scanned/image pages).")
    print(f"[VALIDATE] {len(valid)} non-empty pages retained.")
    return valid

def split_documents(documents: list[Document],
                    chunk_size: int = 1000,
                    chunk_overlap: int = 150) -> list[Document]:
    """Stage 4: split documents into retrieval-sized chunks."""
    splitter = RecursiveCharacterTextSplitter(
        chunk_size=chunk_size,
        chunk_overlap=chunk_overlap,
        separators=["\n\n", "\n", ". ", " ", ""],
        add_start_index=True,      # record each chunk's offset in the source
    )
    chunks = splitter.split_documents(documents)
    print(f"[SPLIT] Produced {len(chunks)} chunks "
          f"(size={chunk_size}, overlap={chunk_overlap}).")
    return chunks

def enrich_metadata(chunks: list[Document], doc_type: str) -> list[Document]:
    """Stage 5: attach a consistent metadata schema to every chunk."""
    today = str(date.today())
    for chunk in chunks:
        chunk.metadata["doc_type"] = doc_type
        chunk.metadata["ingested_on"] = today
        # normalise the source to just the file name for clean citations
        src = chunk.metadata.get("source", "unknown")
        chunk.metadata["file_name"] = Path(src).name

```

```

print(f"[ENRICH] Metadata applied to {len(chunks)} chunks.")
return chunks

def build_chunks(source_dir: str, doc_type: str = "general") -> list[Document]:
    """Run the full ingestion pipeline and return ready-to-embed chunks."""
    documents = load_documents(source_dir)
    documents = validate_documents(documents)
    chunks = split_documents(documents)
    chunks = enrich_metadata(chunks, doc_type)

    # Stage report
    avg_len = (sum(len(c.page_content) for c in chunks) / len(chunks)
               if chunks else 0)
    print(f"[REPORT] {len(chunks)} chunks ready | "
          f"avg chunk length {avg_len:.0f} chars.")
    return chunks

if __name__ == "__main__":
    chunks = build_chunks("data/", doc_type="policy")
    if chunks:
        print("\n--- Sample chunk ---")
        print("Metadata:", chunks[0].metadata)
        print("Content :", chunks[0].page_content[:300], "...")
        # Next: embeddings = ... (Module 3)
        #         vector_store = FAISS.from_documents(chunks, embeddings) (Module 4)

```

2.12 Line-by-Line Code Explanation

The script is deliberately split into small, single-purpose functions because that is how production pipelines are written — each stage is testable and replaceable in isolation.

`load_documents` uses `DirectoryLoader` so the pipeline ingests an entire folder, not one file. `glob="**/*.pdf"` matches every PDF recursively. `loader_cls=PyPDFLoader` tells the directory loader to open each file with `PyPDFLoader`, giving one `Document` per page with `source` and `page` already in metadata. `show_progress=True` prints a progress bar — small comfort on large corpora.

`validate_documents` is the stage beginners omit and professionals never do. It checks each `Document` for non-empty text and discards (and counts) any empty ones. An empty page almost always means a scanned/image page that `PyPDFLoader` could not read. By catching this *at ingestion time* and printing a warning, you turn a silent, baffling failure ("the bot knows nothing about that document") into an explicit, early signal.

`split_documents` configures the `RecursiveCharacterTextSplitter`. `chunk_size` and `chunk_overlap` are function parameters with sensible defaults so they are easy to tune. The custom `separators` list adds `". "` so the splitter can fall back to sentence boundaries before resorting to word boundaries. `add_start_index=True` records, in each chunk's metadata, the character offset where that chunk began in its source document — useful for precise citation and for de-duplication.

`enrich_metadata` applies the metadata schema uniformly. It stamps every chunk with a `doc_type` and an `ingested_on` date, and it normalises `source` (often a long relative path) down to just the file name in a clean `file_name` field, which makes for tidy citations. Doing this in one place guarantees consistency.

`build_chunks` is the orchestrator: it calls the four stages in order and prints a final report with the chunk count and average chunk length. That average is a quick health check — if it is far from your `chunk_size`, something in splitting is off.

The `__main__` block runs the pipeline on the `data/` folder and prints one sample chunk so you can eyeball both its metadata and its text. The trailing comments show exactly where Modules 3 and 4 plug in: the returned `chunks` list is precisely what `FAISS.from_documents(chunks, embeddings)` expects.

BEST PRACTICE

Notice that `build_chunks` returns the chunk list and stops *before* embedding. This separation is intentional: ingestion (load + split + enrich) and indexing (embed + store) are distinct stages that often run in distinct services on distinct schedules. Keeping the function focused on producing clean chunks makes it reusable across every project in this book, regardless of which embedding model or vector store you later choose.

2.13 Common Mistakes

- **Skipping post-load validation.** Loading a scanned PDF "succeeds" but yields empty text; without a validation check the failure is invisible until users complain. Always assert non-empty content.
- **Indexing web-page boilerplate.** Loading a URL without a content filter pulls in menus, footers, and ads as chunks. Filter to the main content region.
- **One splitter for everything.** Using a character splitter on Markdown or code throws away structure. Match the splitter to the content type.
- **Zero overlap.** Saves space, but severs ideas at chunk boundaries and silently harms recall. Use 10–20%.
- **Excessive overlap.** 50%+ overlap multiplies chunk count and cost and floods retrieval with near-duplicates.
- **Treating metadata as optional.** No metadata means no citations, no filtering, no access control, and painful debugging. Design the schema before ingesting.
- **Re-ingesting the whole corpus on every change.** A production pipeline re-processes only the documents that actually changed.
- **Tuning chunk size by intuition alone.** Without measurement you are guessing. Test with representative questions; measure properly in Module 9.

2.14 Best Practices

- **Validate immediately after loading.** Catch empty or failed extractions at ingestion, not in production.
- **Match the splitter to the content.** Recursive for prose, header-aware for Markdown, language-aware for code, semantic for high-value corpora.
- **Default to ~1000-character chunks with 10-20% overlap**, then tune by measurement.
- **Design a consistent metadata schema before ingesting**, and apply it to every chunk.
- **Filter web content** to its main region before indexing.
- **Separate ingestion from indexing.** Produce clean chunks first; embed and store as a distinct stage.
- **Make ingestion incremental and observable.** Re-process only what changed; log counts, sources, and timestamps.
- **Use `lazy_load()` for very large sources** to avoid loading everything into memory at once.

2.15 Real-World Applications

- **Multi-PDF knowledge assistants.** `DirectoryLoader` ingests a whole folder of policies or manuals; consistent metadata lets users filter by document and receive citations.
- **Documentation chatbots.** Markdown-aware splitting keeps each documentation section coherent and tags every chunk with its section path.
- **"Chat with a repository" tools.** Language-aware code splitting keeps functions whole so retrieved chunks are complete logic units.
- **Website Q&A bots.** `WebBaseLoader` with content filtering ingests only the meaningful article text of each page.
- **YouTube Q&A assistants.** A transcript loader turns video subtitles into documents that are then chunked like any other text.
- **Compliance and legal search.** Rich metadata (jurisdiction, date, document type) enables precise, filtered retrieval over large regulated corpora.

2.16 Mini Project: A Chunking Inspector

Understanding chunking deeply means *seeing* it. Build a small inspector tool:

1. Take a PDF you own and run it through `build_chunks` from `ingest.py`.

2. Print, for the first ten chunks: the chunk index, its character length, its metadata, and its first and last 80 characters. Observe how consecutive chunks overlap.
3. **Experiment with size.** Re-run with `chunk_size` set to 300, then 1000, then 2500. For each, record the total chunk count and the average length. Write two sentences on how chunk size changes the *shape* of the knowledge base.
4. **Experiment with overlap.** Fix `chunk_size=1000` and try `chunk_overlap` of 0, 150, and 500. Inspect the boundary text between chunk 1 and chunk 2 in each case and describe what you see.
5. **Compare splitters.** Run the same document through `RecursiveCharacterTextSplitter` and `CharacterTextSplitter` with identical size settings. Find one place where the recursive splitter produced a cleaner boundary and explain why.

This inspector makes chunking concrete instead of abstract — and it is a genuinely useful diagnostic tool to keep for future projects.

2.17 Chapter Summary

- **Document processing** is the front of the indexing pipeline: **loading** (extracting text + metadata from raw sources) and **splitting** (cutting that text into retrieval-sized chunks). It is where most RAG quality — and most RAG bugs — originate.
- The `Document` object (with `page_content` and `metadata`) is LangChain's universal currency; every loader produces it and every splitter consumes it, which is what makes pipelines composable.
- **Document loaders** convert each source type into `Document` objects: `PyPDFLoader` and friends for PDFs (beware scanned PDFs producing empty text), `WebBaseLoader` for web pages (always filter to main content), `CSVLoader` / `JSONLoader` for structured data, and `DirectoryLoader` for whole folders.
- **Splitting is mandatory** because of finite context windows, the *one chunk → one vector* precision rule, embedding-model input limits, and cost.
- Splitter choices: `CharacterTextSplitter` (simplest), `RecursiveCharacterTextSplitter` (the structure-aware default), `MarkdownHeaderTextSplitter` (section-aware), `from_language` recursive splitting (code-aware), and `SemanticChunker` (meaning-aware).
- The recursive splitter works in two stages: **recursively split** at the best available separator, then **merge** fragments back up to `chunk_size` with overlap.
- **Chunk size** is a precision-versus-context trade-off; ~800–1200 characters is a reliable default. **Chunk overlap** of 10–20% prevents ideas being severed at boundaries. Tune both by measurement.
- **Metadata** enables citations, filtering, access control, and lifecycle management. Design a consistent schema before ingesting and apply it to every chunk.

- A **production ingestion pipeline** adds validation, incrementality, de-duplication, and logging on top of the same loaders and splitters — and you built a reusable `ingest.py` embodying these principles.

2.18 Practice Exercises

1. **Conceptual.** Explain in your own words why a chunk that is too large hurts retrieval, using the phrase "one chunk, one vector." Keep it under 120 words.
2. **Loader choice.** For each source, name the loader you would use and one risk to check for: (a) a folder of 200 mixed-format files; (b) a single scanned contract; (c) a documentation website; (d) a CSV of customer records.
3. **Splitter choice.** You are building an assistant over a software project that contains Python files, Markdown docs, and PDF manuals. Describe the splitting strategy you would apply to each, and why.
4. **Size experiment.** Using the mini-project inspector, ingest one document at chunk sizes 300, 1000, and 2500. Report the chunk counts and write three sentences interpreting the results.
5. **Overlap experiment.** Fix `chunk_size=800`. Find a sentence in your document that gets split across a chunk boundary at `chunk_overlap=0`. Show that `chunk_overlap=150` keeps that sentence whole in at least one chunk.
6. **Metadata design.** Design a metadata schema (list the fields and their types) for a RAG system serving a company with HR, Finance, and Engineering departments, where some documents are confidential. Explain what each field enables.
7. **Pipeline extension.** Extend `ingest.py` with a stage 6 de-duplication step that drops chunks whose `page_content` is identical to one already kept. Report how many duplicates it removes on a corpus that includes two versions of the same document.

2.19 Interview Questions

BEGINNER

Q1. What is a document loader, and what does it produce?

A component that reads a data source (PDF, web page, CSV, etc.) and converts it into standardised `Document` objects, each holding extracted text (`page_content`) and origin information (`metadata`).

Q2. Why must documents be split into chunks before embedding?

Because the LLM's context window is finite, because an embedding is one vector per chunk so smaller focused chunks retrieve more precisely, because embedding models have input limits, and because oversized chunks raise cost and add noise.

Q3. What is chunk overlap and why is it used?

It is text shared between consecutive chunks. It solves the boundary problem: without overlap, an idea sitting on a chunk boundary is split in half and becomes unretrievable; overlap ensures boundary text appears in full in at least one chunk.

Q4. What is the difference between `CharacterTextSplitter` and `RecursiveCharacterTextSplitter` ?

The character splitter uses a single separator and cannot gracefully break an oversized piece. The recursive splitter tries an ordered list of separators, falling back from paragraphs to lines to words to characters, so it respects natural structure and avoids crude mid-word cuts.

INTERMEDIATE**Q5. How would you decide on a chunk size for a new corpus?**

Start from a content-appropriate default (~1000 characters for general prose), build the pipeline, then run representative test questions. Vague, unfocused answers suggest chunks are too large; fragmented answers missing context suggest they are too small. Make it rigorous with retrieval-quality metrics (RAGAS, Module 9). The aim is one self-contained idea with enough context to stand alone.

Q6. Explain how the recursive splitter works internally.

Two stages. First it recursively splits: it tries the first separator (paragraph breaks); any piece still over `chunk_size` is split on the next separator (lines), then words, then characters. Second it merges adjacent small pieces back up toward `chunk_size` and stitches in the requested overlap. The effect is to break text at the most natural boundary that satisfies the size limit.

Q7. Why is metadata important in a RAG system? Give three uses.

Source citations (traceability and trust), metadata filtering (restricting search to a relevant subset, improving precision and speed), and access control (returning only chunks a user is permitted to see). It also enables lifecycle management — updating or deleting all chunks from a changed document.

Q8. When would you use a Markdown-aware or code-aware splitter instead of the recursive character splitter?

Markdown-aware splitting when the content has header structure you want to preserve and capture in metadata (wikis, technical docs). Code-aware splitting when ingesting source code, so functions and classes stay intact and a retrieved chunk is a complete logic unit rather than a fragment.

ADVANCED**Q9. What is semantic chunking, and what are its trade-offs versus recursive character splitting?**

Semantic chunking places boundaries by meaning: it embeds sentences and cuts where consecutive sentences shift topic sharply, yielding maximally coherent single-topic chunks. The trade-off is cost and speed — it must embed every sentence at ingestion time. Recursive character splitting is far cheaper and "good enough" for most corpora; reserve semantic chunking for high-value corpora where retrieval quality justifies the extra ingestion cost.

Q10. Your RAG system gives correct-sounding but incomplete answers. How might chunking be the cause, and how would you investigate?

Likely causes: chunks too small (the full answer is spread across several chunks and only one is retrieved), zero or insufficient overlap (an idea was severed at a boundary), or a structure-blind splitter that cut mid-section. Investigate by inspecting the actual retrieved chunks for the failing query; if the answer text is fragmented across chunk boundaries, increase chunk size or overlap, or switch to a structure-aware splitter, then re-measure.

Q11. How does a production ingestion pipeline differ from a prototype script?

It uses the same loaders and splitters but adds: post-load validation (catch empty/failed extractions early), incremental processing (re-ingest only changed documents), deduplication, lazy loading for memory safety on large sources, a consistent enforced metadata schema, and thorough logging so each ingestion run is auditable.

SCENARIO-BASED**Q12. A client's RAG bot returns nothing useful about three specific PDFs, though it works for all others. Those three are scanned contracts. What happened and how do you fix it?**

The three PDFs are image-only (scanned), so a standard text-based loader like `PyPDFLoader` extracted empty or near-empty text — `.load()` succeeded silently but indexed nothing. The fix is an OCR-capable loader for those files. The deeper fix is a validation stage that asserts non-empty content right after loading, so this class of failure is flagged at ingestion instead of discovered by users.

Q13. You must build a RAG system over a knowledge base where roughly 5% of documents change every day. How do you design ingestion?

Make ingestion incremental, not full-rebuild. Track each document's identity and a content hash or last-modified timestamp. On each run, ingest only new or changed documents; for a changed document, delete its existing chunks (found via a stable `source` metadata key) and re-ingest just that document. This requires consistent per-document metadata so chunks can be located and replaced, and it keeps daily ingestion cheap and fast instead of re-embedding the entire corpus every day.

END OF MODULE 2

You can now turn any messy real-world source into clean, well-sized, well-labelled chunks, and you have a reusable ingestion pipeline to prove it. But those chunks are still plain text.

Module 3 — Embeddings and Vector Representations takes the next step: how text is converted into the numerical vectors that make semantic search possible, how similarity is measured, how to choose an embedding model, and how to build practical embedding pipelines with Gemini. That is where your chunks finally become searchable by meaning.